

PROJECT REPORT

on

AI-Based Automotive Review and Customer Sentiment Analytics

Aspect-Based Sentiment Analysis of Automotive Reviews using Fine-Tuned BERT

SUBMITTED BY

Lopamudra Roy

Enrollment No.: 12023002001195

Department of Computer Science and Engineering

Institute of Engineering and Management, Kolkata

Academic Session: 2023–2027

Abstract

Online automotive reviews contain valuable information about how customers perceive the engine, battery, safety features, comfort, service quality, infotainment and price of a vehicle. However, a single review usually mentions several of these aspects with different opinions, so a single overall sentiment score hides the details that manufacturers and buyers actually need. This project, **AI-Based Automotive Review and Customer Sentiment Analytics**, addresses the problem through aspect-based sentiment analysis (ABSA).

The system has three parts. First, a keyword-driven aspect extractor detects which of nine automotive aspects (Vehicle, Engine, Battery, Mileage, Safety, Comfort, Service, Infotainment and Price) are discussed in a review. Second, a bert-base-uncased transformer is fine-tuned with the Hugging Face Trainer API on aspect-prefixed inputs of the form “aspect: review” to classify sentiment as Negative, Neutral or Positive. Third, a Streamlit web dashboard called **AutoInsight AI** lets a user paste a review and instantly see aspect tags with sentiment, together with an overall sentiment score.

A balanced dataset of 216 labelled review sentences ($9 \text{ aspects} \times 3 \text{ sentiments} \times 8 \text{ samples}$) was cleaned, de-duplicated and divided into 151 training, 32 validation and 33 test samples using stratified sampling. The keyword extractor achieved 88.0% recall of the labelled aspect on the dataset, and a TF-IDF with logistic regression baseline reached 63.6% accuracy on the held-out test set, providing a reference point for the fine-tuned BERT model. The report also discusses the limitations of the small, template-like dataset and outlines future work including larger real-world corpora, learned aspect extraction and deployment as a service.

Keywords: Sentiment Analysis, Aspect-Based Sentiment Analysis, BERT, Transformers, Automotive Reviews, Natural Language Processing, Streamlit.

Table of Contents

Abstract.....	1
1. Introduction.....	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives.....	4
1.4 Scope and Limitations	5
1.5 Organisation of the Report	5
2. Background and Literature Review	6
2.1 Sentiment Analysis.....	6
2.2 Approaches to Sentiment Classification.....	6
2.3 Aspect-Based Sentiment Analysis.....	6
2.4 Tooling Background.....	7
2.5 Research Gap and Motivation	7
3. Tools and Environment	8
3.1 Model and Hardware Environment	8
3.2 Project Directory Layout	9
4. System Design and Methodology	10
4.1 Overall Architecture.....	10
4.2 Dataset Design.....	10
4.3 Preprocessing Design	11
4.4 Label Encoding and Model Input Format	11
4.5 Data Splitting Strategy	11
4.6 Model Design	12
4.7 Aspect Extraction Design.....	12
4.8 Dashboard Design	13
4.9 Evaluation Metrics	13
5. Implementation	14
5.1 Data Preprocessing (preprocessing.py)	14
5.2 Dataset Preparation (prepare_data.py)	14
5.3 BERT Fine-Tuning (train_bert.py).....	14
5.4 Aspect Extractor (aspect_extractor.py)	15
5.5 Command-Line Inference (predict.py).....	15
5.6 Web Application (app.py)	16
6. Testing and Results	17
6.1 Aspect Extraction Evaluation.....	17
6.2 Sentiment Classification: Baseline.....	17
6.3 Sentiment Classification: Fine-Tuned BERT.....	18
6.4 Functional Testing of Aspect Detection.....	18
6.5 End-to-End Test with the Dashboard	19
6.6 Summary of Results	20
7. Discussion.....	21
7.1 Interpretation of Findings.....	21
7.2 Effect of Sentence-Level Context	21
7.3 Limitations Observed	21
7.4 Practical Implications.....	22
8. Conclusion and Future Scope	23
8.1 Conclusion.....	23

8.2 Future Scope.....	23
References.....	24
Appendix.....	25
Appendix A: Sample Dataset Rows	25
Appendix B: aspect_extractor.py (Complete Listing).....	25
Appendix C: prepare_data.py (Complete Listing)	27
Appendix D: train_bert.py (Training Configuration and Trainer)	28
Appendix E: predict.py (Inference Functions)	29
Appendix F: requirements.txt.....	30

1. Introduction

1.1 Background

The automotive industry has become strongly customer-driven. Before purchasing a car, most buyers read reviews on manufacturer websites, dealer portals, social media and video platforms. After purchase, owners share their experience about performance, comfort, running cost and after-sales service. This continuous stream of opinions is a rich source of feedback, but it is unstructured text and grows too fast to be read manually.

Modern vehicles are also becoming software-defined. Electric vehicles bring new concerns such as battery range and charging time, while In-Vehicle Infotainment (IVI) systems—touchscreens, navigation, audio and phone connectivity—have become a major factor in the driver experience, and, through distraction and usability, in safety. Customer reviews therefore mix traditional aspects (engine, mileage, comfort) with technology-related aspects (battery, infotainment) in the same sentence.

1.2 Problem Statement

Conventional sentiment analysis assigns one polarity to an entire review. Consider the review “The car is expensive but the safety features are excellent.” It is negative about price and positive about safety; labelling it with one overall polarity loses information that is essential for a product manager (who must decide what to fix) and for a buyer (who cares about specific features). The problem addressed in this project is: **given a free-text automotive review, identify the automotive aspects discussed and predict the sentiment expressed towards each aspect.**

1.3 Objectives

- To build a labelled dataset of automotive review sentences covering nine aspects and three sentiment classes.
- To design a text preprocessing and dataset-preparation pipeline with stratified train, validation and test splits.
- To fine-tune a pre-trained BERT model for three-class aspect-level sentiment classification.
- To implement an aspect extraction module that maps review text to automotive aspects.
- To integrate the modules into an interactive web dashboard (AutoInsight AI) that presents aspect tags, sentiment and an overall score.

- To evaluate the components and compare the approach with a classical machine-learning baseline.

1.4 Scope and Limitations

The scope of the project is aspect-level sentiment analysis of English-language, sentence-level automotive review text through a software application. The following limitations apply:

- **Software-only prototype.** The system analyses text supplied by the user. It is not connected to any real vehicle, infotainment head unit or vehicle bus, and no live data is collected from review websites.
- **Small, curated dataset.** The training data consists of 216 short review sentences with balanced classes. Results on this dataset cannot be assumed to transfer to long, noisy, real-world reviews.
- **Fixed aspect set.** Aspect detection is keyword-based and limited to nine predefined aspects; implicit aspects and unseen vocabulary are not handled.
- **Three-class polarity only.** Fine-grained emotions, sarcasm and mixed sentiment within one aspect are outside the scope.
- **Compute.** Training was configured to run on CPU only, which restricts model size, batch size and the number of experiments.

1.5 Organisation of the Report

Chapter 2 reviews the background and related work. Chapter 3 lists the tools and environment. Chapter 4 describes the system design and Chapter 5 the implementation. Chapter 6 presents testing and results, Chapter 7 discusses the findings, and Chapter 8 concludes with future scope. References and an appendix with code listings and usage instructions follow.

2. Background and Literature Review

2.1 Sentiment Analysis

Sentiment analysis, or opinion mining, is the computational study of people’s opinions, attitudes and emotions expressed in text [1]. It is commonly studied at three levels: **document level** (one polarity for a whole review), **sentence level** (one polarity per sentence) and **aspect level** (polarity towards each entity or feature mentioned). Early work on customer reviews by Hu and Liu extracted product features and summarised the opinions about each feature [2], which is the origin of what is now called aspect-based sentiment analysis.

2.2 Approaches to Sentiment Classification

2.2.1 *Lexicon and rule-based methods*

Lexicon-based methods count positive and negative words from a dictionary and apply rules for negation and intensifiers. Tools such as VADER [3] are fast and need no training data, but they struggle with domain-specific vocabulary (for example, “quiet” is positive for a cabin but may be negative for an audio system) and with context.

2.2.2 *Classical machine learning*

Supervised classifiers such as Naive Bayes, Support Vector Machines and Logistic Regression operate on bag-of-words or TF-IDF features. They are simple and efficient and form a standard baseline, but they ignore word order and context, so “not good” and “good” differ only by the presence of one more feature.

2.2.3 *Deep learning and transformers*

Recurrent networks (LSTM/GRU) model word order but process text sequentially. The Transformer architecture [4] replaced recurrence with self-attention, allowing every word to attend to every other word in the sentence. BERT (Bidirectional Encoder Representations from Transformers) [5] pre-trains a Transformer encoder on large text corpora using masked language modelling and next-sentence prediction, and can then be fine-tuned with a small task-specific output layer. This transfer-learning approach achieves strong results on small labelled datasets, which makes it attractive for a domain such as automotive reviews where labelled data is limited.

2.3 Aspect-Based Sentiment Analysis

ABSA was formalised as a shared task in SemEval-2014 Task 4, using restaurant and laptop reviews [6]. The task has two main sub-problems: **aspect extraction** (find the aspects

mentioned) and **aspect sentiment classification** (decide the polarity for each aspect). Sun, Huang and Qiu showed that BERT can be applied to aspect sentiment classification by converting the task into a sentence-pair problem, where an auxiliary sentence naming the aspect is paired with the review [7]. The present project follows the same principle in a simplified form: the aspect name is prepended to the review, producing the input “aspect: review”, so that one review can yield different predictions for different aspects.

2.4 Tooling Background

The Hugging Face Transformers library [8] provides pre-trained models, tokenizers and the Trainer API used here. PyTorch [9] is the underlying tensor and deep-learning framework, scikit-learn [10] supplies data splitting and evaluation metrics, and Streamlit provides a lightweight way to build interactive data applications in Python.

2.5 Research Gap and Motivation

Most published ABSA work targets restaurants and consumer electronics. Automotive reviews have their own vocabulary (kmpl, ADAS, legroom, range anxiety) and combine mechanical, electrical and software concerns. This project contributes a compact, end-to-end, reproducible pipeline for the automotive domain—from data preparation through fine-tuning to an interactive dashboard—suitable for study and extension.

3. Tools and Environment

The project is implemented entirely in Python. Table 3.1 lists the libraries used, taken from the project's `requirements.txt`, and their roles.

Table 3.1: Software tools and libraries

Tool / Library	Purpose in this project
Python 3	Primary programming language for all scripts
pandas, NumPy	Loading CSV files, cleaning, de-duplication, array operations
scikit-learn	Stratified train/validation/test split; accuracy, precision, recall and F1 metrics; TF-IDF and logistic regression baseline
PyTorch (torch)	Deep-learning backend for BERT training and inference
transformers	<code>AutoTokenizer</code> , <code>AutoModelForSequenceClassification</code> , <code>Trainer</code> and <code>TrainingArguments</code>
datasets	Conversion of pandas DataFrames into Hugging Face <code>Dataset</code> objects and batched tokenization
accelerate	Training support library required by the Hugging Face <code>Trainer</code>
Streamlit	Web dashboard (AutoInsight AI)
Plotly	Donut chart of the sentiment score in the dashboard
matplotlib, seaborn	Plotting and analysis of results
NLTK, spaCy	Natural language processing utilities (listed as project dependencies for text processing)
openpyxl	Reading and writing Excel files
re (standard library)	Regular expressions for text cleaning and keyword matching

3.1 Model and Hardware Environment

- **Pre-trained model:** bert-base-uncased (12-layer Transformer encoder, about 110 million parameters, lower-cased vocabulary).
- **Training device:** CPU only. The `TrainingArguments` in `train_bert.py` set `use_cpu=True`, so no GPU is required to reproduce the training.
- **Editor / run mode:** scripts are run from the command line; the dashboard is launched with `streamlit run app.py` and viewed in a web browser.

3.2 Project Directory Layout

```
project/
|-- app.py                # Streamlit dashboard (AutoInsight AI)
|-- aspect_extractor.py   # Keyword-based aspect extraction
|-- predict.py            # Command-line aspect-level prediction
|-- preprocessing.py      # Cleaning of raw reviews
|-- prepare_data.py       # Train / validation / test split
|-- train_bert.py         # BERT fine-tuning and evaluation
|-- requirements.txt
|-- data/
|   |-- automotive_reviews.csv # raw labelled dataset (216 rows)
|   |-- processed_reviews.csv  # cleaned dataset
|   |-- train.csv  validation.csv  test.csv
|-- model/
|   |-- bert_automotive_sentiment/ # saved fine-tuned model
```

4. System Design and Methodology

4.1 Overall Architecture

The system is organised as two pipelines that share the same data format (Figure 4.1). The **training pipeline** converts the raw labelled reviews into a fine-tuned BERT model. The **inference pipeline** takes a new review, detects its aspects, classifies the sentiment for each aspect using the saved model, and presents the results in the dashboard.

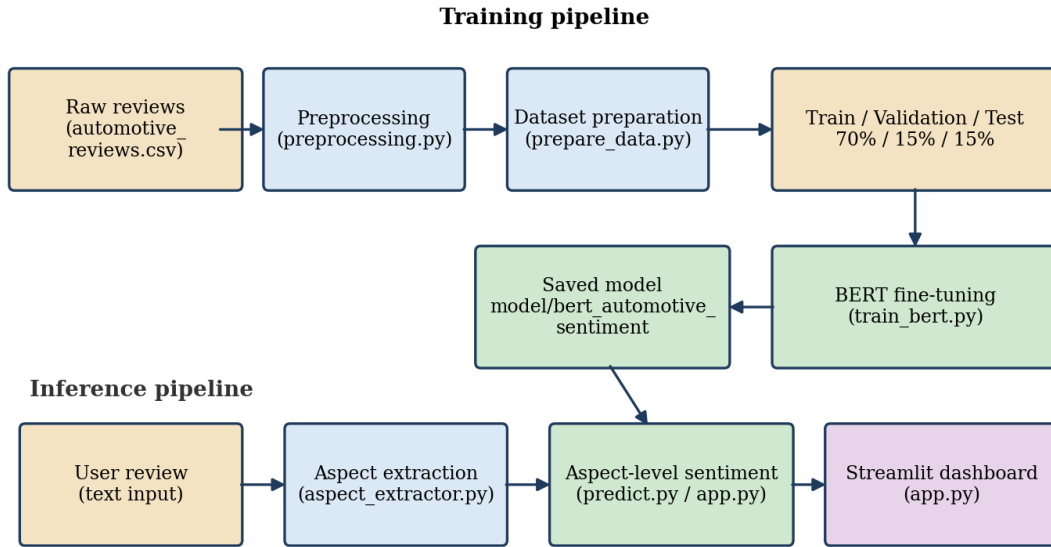


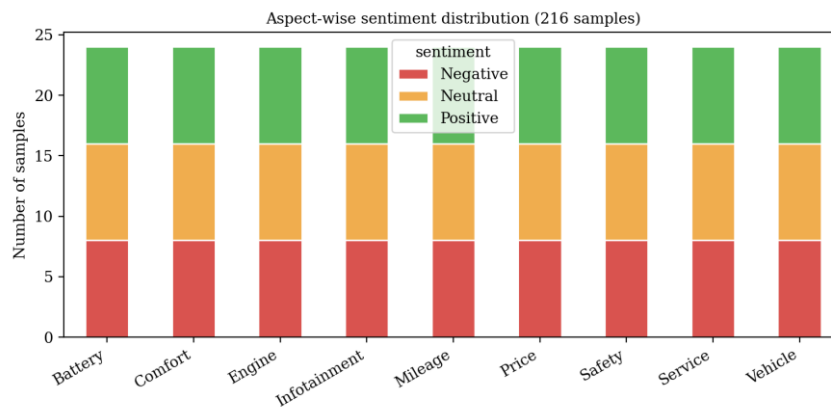
Figure 4.1: Overall system architecture (training and inference pipelines)

4.2 Dataset Design

The dataset `automotive_reviews.csv` contains three columns: `review` (a short review sentence), `aspect` (one of nine automotive aspects) and `sentiment` (Negative, Neutral or Positive). Each of the nine aspects has exactly eight samples per sentiment class, giving 72 samples per class and 216 in total. Balanced classes avoid the need for class weighting and make accuracy a meaningful metric. Table 4.1 summarises the dataset and Figure 4.2 shows the distribution.

Table 4.1: Dataset summary

Property	Value
Total samples	216 (no missing values, no duplicate rows)
Aspects (9)	Vehicle, Engine, Battery, Mileage, Safety, Comfort, Service, Infotainment, Price
Sentiment classes (3)	Negative, Neutral, Positive
Samples per aspect	24 (8 per sentiment class)
Samples per class	72 Negative, 72 Neutral, 72 Positive
Review length	4–10 words; mean about 5.9 words
Unique review texts	216 (every review is distinct)

*Figure 4.2: Aspect-wise sentiment distribution of the dataset*

4.3 Preprocessing Design

The preprocessing stage removes rows with missing review, aspect or sentiment values, drops duplicate rows, and normalises the review text into a `clean_review` column: conversion to lower case, removal of every character that is not a letter, digit or whitespace, collapsing of repeated whitespace, and trimming. Rows whose cleaned text is empty are discarded. Because BERT’s tokenizer is uncased, lower-casing loses no information for the chosen model.

4.4 Label Encoding and Model Input Format

Sentiment labels are mapped to integers: **Negative = 0**, **Neutral = 1**, **Positive = 2**. The model input is constructed by concatenating the aspect and the cleaned review with a colon:

```
bert_text = aspect + ": " + clean_review
# e.g. "price: the price is higher than expected" -> label 0 (Negative)
```

Providing the aspect as a text prefix lets the same review sentence be classified differently for different aspects, without changing the model architecture. The BERT tokenizer converts this

string into token IDs, adds the special [CLS] and [SEP] tokens, and truncates to at most 128 tokens.

4.5 Data Splitting Strategy

The processed data is divided using two successive calls to `train_test_split` with `random_state=42` and stratification on the label: 70% for training and 30% held aside, and then the held-aside portion is divided equally into 15% validation and 15% test. Stratification keeps the class proportions almost identical in all three subsets (Table 4.2, Figure 4.3), and the fixed seed makes the split reproducible.

Table 4.2: Data split

Subset	Samples	Negative	Neutral	Positive
Training (70%)	151	51	50	50
Validation (15%)	32	10	11	11
Test (15%)	33	11	11	11
Total	216	72	72	72



Figure 4.3: Class distribution across training, validation and test sets

No review text appears in more than one subset (verified by comparing the `clean_review` columns of the three files), so there is no direct text leakage between training and evaluation data.

4.6 Model Design

BERT-base consists of an embedding layer followed by 12 Transformer encoder layers with 12 attention heads and a hidden size of 768. For classification, the final hidden state of the [CLS] token is passed through a dropout layer and a linear layer with three outputs (one per sentiment class), which is added by `AutoModelForSequenceClassification` with `num_labels=3`. During fine-tuning, all parameters are updated using the cross-entropy loss

between the predicted class scores and the true label. At inference time, a softmax over the three logits gives class probabilities and the arg-max gives the predicted sentiment.

4.7 Aspect Extraction Design

Aspect extraction uses a dictionary that maps each of the nine aspects to a list of trigger keywords (for example, Battery: battery, range, charging; Safety: safety, airbag, brake, ADAS). A review is scanned for each keyword using a regular expression with word boundaries (`\bkeyword\b`), and an aspect is reported once as soon as any of its keywords is found. This design is transparent, needs no training data and is fast, at the cost of missing aspects that are expressed with words outside the dictionary.

4.8 Dashboard Design

The Streamlit dashboard uses a dark theme with a sidebar for navigation (Dashboard, Reviews, Analytics, Aspects, Models, Alerts, Settings). The main Dashboard page contains: (i) a headline and short description; (ii) three feature cards (BERT Intelligence, Aspect-Level Analysis, Instant Insights); (iii) a text area for the customer review and an “Analyze Review” button; (iv) coloured aspect tags showing the sentiment of each detected aspect; and (v) a “Sentiment Score Overview” donut chart with an overall score.

4.9 Evaluation Metrics

Model performance is measured with the metrics computed in `train_bert.py` using `scikit-learn`, with weighted averaging across classes:

- **Accuracy** = (correct predictions) / (all predictions).
- **Precision** = $TP / (TP + FP)$ for each class, averaged with class-frequency weights.
- **Recall** = $TP / (TP + FN)$ for each class, averaged with class-frequency weights.
- **F1-score** = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$; the weighted F1 is also the criterion used to select the best checkpoint.

5. Implementation

5.1 Data Preprocessing (preprocessing.py)

The script reads `data/automotive_reviews.csv`, drops rows with missing values in the review, aspect or sentiment columns, removes duplicates, applies the `clean_text` function and writes `data/processed_reviews.csv`. It also prints the sentiment and aspect distributions as a sanity check. The core cleaning function is:

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"^[a-zA-Z0-9\s]", "", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

For the supplied dataset, the shape remained 216×3 before and 216×4 after preprocessing (a new `clean_review` column was added), because the raw file contained no missing values or duplicates.

5.2 Dataset Preparation (prepare_data.py)

This script builds the BERT input string, maps sentiment names to integer labels and performs the stratified 70/15/15 split described in Section 4.5. It saves `train.csv`, `validation.csv` and `test.csv`, each containing the columns review, aspect, sentiment, clean_review, bert_text and label.

```
df["bert_text"] = df["aspect"] + ": " + df["clean_review"]
label_mapping = {"Negative": 0, "Neutral": 1, "Positive": 2}
df["label"] = df["sentiment"].map(label_mapping)

train_df, temp_df = train_test_split(
    df, test_size=0.30, random_state=42, stratify=df["label"])
validation_df, test_df = train_test_split(
    temp_df, test_size=0.50, random_state=42, stratify=temp_df["label"])
```

5.3 BERT Fine-Tuning (train_bert.py)

The training script follows the standard Hugging Face workflow: load the three CSV files, convert them to Dataset objects, tokenize with `bert-base-uncased` (truncation at 128 tokens), load `AutoModelForSequenceClassification` with three labels and the `id2label/label2id` mappings, pad each batch dynamically with `DataCollatorWithPadding`, and train with `Trainer`. After training, the model is evaluated on the test set and both the model and tokenizer are saved to `model/bert_automotive_sentiment`. Table 5.1 lists the training configuration.

Table 5.1: Training hyper-parameters

Parameter	Value
Pre-trained checkpoint	bert-base-uncased
Number of labels	3 (Negative, Neutral, Positive)
Maximum sequence length	128 tokens
Learning rate	2e-5
Batch size (train / eval)	8 / 8
Epochs	3 (about 19 steps per epoch, 57 optimisation steps in total)
Weight decay	0.01
Evaluation and checkpoint strategy	At the end of every epoch
Model selection	Best weighted F1 on the validation set (load_best_model_at_end=True)
Metrics reported	Accuracy, weighted precision, weighted recall, weighted F1
Device	CPU (use_cpu=True)

Because only 151 samples are used for training, the entire fine-tuning is short. The learning rate of 2e-5 is within the range recommended for BERT fine-tuning and the validation set is used only for checkpoint selection, while the test set is used once for the final evaluation.

5.4 Aspect Extractor (aspect_extractor.py)

The module holds the ASPECT_KEYWORDS dictionary and the function `extract_aspects(text)`, which lower-cases the text and searches for each keyword with a word-boundary regular expression. The full listing is given in Appendix B.

```
def extract_aspects(text):
    text = text.lower()
    found_aspects = []
    for aspect, keywords in ASPECT_KEYWORDS.items():
        for keyword in keywords:
            pattern = r"\b" + re.escape(keyword) + r"\b"
            if re.search(pattern, text):
                found_aspects.append(aspect)
                break
    return found_aspects
```

5.5 Command-Line Inference (predict.py)

`predict.py` loads the saved tokenizer and model once, sets the model to evaluation mode, and defines `analyze_review(review)`. For each aspect returned by `extract_aspects`, it builds the input `aspect + ": " + review`, tokenizes it (maximum 128 tokens), runs a forward pass

without gradient tracking and returns the label with the highest logit. The output is a list of {aspect, sentiment} pairs, printed as “Aspect → Sentiment”.

5.6 Web Application (app.py)

The Streamlit application, AutoInsight AI, wraps the same idea in an interactive interface. Key implementation points are:

- **Caching.** `st.cache_data` caches the dataset and `st.cache_resource` caches the tokenizer and model so they are loaded only once per session.
- **Session state.** The current page, review text and analysis results are stored in `st.session_state`, so the interface keeps its state across reruns.
- **Sentence-level context.** The review is split into sentences at “.”, “!” and “?”. For each detected aspect, only the sentences containing that aspect’s keywords are passed to the model, which helps when one review contains opposite opinions about different aspects.
- **Hybrid prediction.** The app runs the fine-tuned BERT model on the “aspect: context sentence” input. A small sentiment lexicon of positive and negative words, with a simple negation rule, is used as a cross-check: if the lexicon strongly disagrees with a BERT “Negative” or “Positive” output, the lexicon polarity is used. If the model directory cannot be loaded, the lexicon alone provides a fallback so the dashboard remains usable.
- **Visualisation.** Aspect tags are rendered as coloured chips (green for Positive, yellow for Neutral, red for Negative) and the overall score is drawn as a Plotly donut chart, with positive share shown as a percentage.
- **Navigation.** A sidebar provides the pages Dashboard, Reviews, Analytics, Aspects, Models, Alerts and Settings.

Running the system

```
pip install -r requirements.txt
python preprocessing.py          # data/automotive_reviews.csv -> processed_reviews.csv
python prepare_data.py          # creates train.csv, validation.csv, test.csv
python train_bert.py            # fine-tunes BERT and saves the model
python predict.py               # optional: command-line prediction
streamlit run app.py            # launches the AutoInsight AI dashboard
```

6. Testing and Results

Testing was carried out at three levels: component-level evaluation of the aspect extractor, quantitative evaluation of sentiment classification on the held-out test set, and functional testing of the complete dashboard with sample reviews.

6.1 Aspect Extraction Evaluation

The keyword extractor was run on all 216 review sentences and its output was compared with the labelled aspect. Table 6.1 and Figure 6.1 summarise the results.

Table 6.1: Aspect extractor performance on the 216-sample dataset

Measure	Result
Labelled aspect found among detected aspects (recall)	190 / 216 = 88.0%
Exactly one aspect detected and it equals the label	131 / 216 = 60.6%
More than one aspect detected	59 / 216 = 27.3%
No aspect detected	8 / 216 = 3.7%

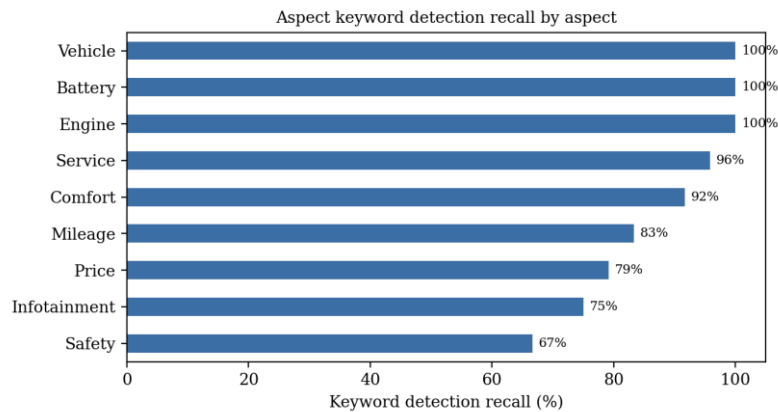


Figure 6.1: Recall of the keyword extractor for each aspect

Engine, Battery and Vehicle reach 100% recall because their vocabularies are distinctive. Safety (66.7%) and Infotainment (75.0%) are lowest because reviews often use words outside the dictionary, such as the plural “airbags”, “rear camera”, “stability control”, “USB”, “speakers” or “connectivity”. Most multi-aspect detections are caused by the generic word “car” or “vehicle”, which triggers the Vehicle aspect in addition to the true aspect (for example, “The car has six airbags” yields Vehicle instead of Safety). These are natural consequences of the dataset labelling convention—each sentence has a single labelled aspect—while real reviews genuinely contain several aspects.

6.2 Sentiment Classification: Baseline

To give the BERT results context, a classical baseline was trained on exactly the same split: TF-IDF features (unigrams and bigrams) computed on the `bert_text` field, followed by a logistic regression classifier. Table 6.2 shows its performance and Table 6.3 the confusion matrix on the test set.

Table 6.2: TF-IDF + Logistic Regression baseline (weighted metrics)

Subset	Accuracy	Precision	Recall	F1-score
Validation (32)	0.719	0.720	0.719	0.718
Test (33)	0.636	0.633	0.636	0.631

Table 6.3: Baseline confusion matrix on the test set (rows: actual, columns: predicted)

Actual \ Predicted	Negative	Neutral	Positive
Negative	5	2	4
Neutral	2	8	1
Positive	2	1	8

The baseline classifies the neutral class best, but frequently confuses Negative with Positive, which indicates that word counts alone are not enough when sentences are short and share words such as “price”, “engine” or “system”.

6.3 Sentiment Classification: Fine-Tuned BERT

The fine-tuned BERT model is evaluated by `train_bert.py`, which prints the validation metrics after every epoch and the final test metrics after training. Table 6.4 is provided for these values. The highlighted cells must be filled in from the console output of `train_bert.py` (keys `eval_accuracy`, `eval_precision`, `eval_recall`, `eval_f1`) after training on the final data.

Table 6.4: Fine-tuned BERT performance (weighted metrics)

Subset	Accuracy	Precision	Recall	F1-score
Validation (32)	[fill]	[fill]	[fill]	[fill]
Test (33)	[fill]	[fill]	[fill]	[fill]

Because the test set contains only 33 samples, each sample changes accuracy by about 3 percentage points; the numbers should therefore be interpreted as indicative rather than definitive.

6.4 Functional Testing of Aspect Detection

Four sample reviews included in `aspect_extractor.py` were used as a functional test. Table 6.5 lists the aspects detected by the module and the sentiment a human reader would expect for each.

Table 6.5: Functional test of the aspect extractor

Test review	Detected aspects	Expected sentiment (manual)
The battery range is excellent.	Battery	Battery: Positive
The engine is powerful but the mileage is poor.	Engine, Mileage	Engine: Positive; Mileage: Negative
The seats are comfortable and the infotainment system is easy to use.	Comfort, Infotainment	Comfort: Positive; Infotainment: Positive
The car is expensive but the safety features are excellent.	Vehicle, Safety, Price	Price: Negative; Safety: Positive (Vehicle is a spurious detection)

6.5 End-to-End Test with the Dashboard

A longer, multi-sentence review was entered into the AutoInsight AI dashboard:

“The exterior vehicle design looks modern and sleek. However, the maintenance cost is ridiculously expensive, and the customer service at the dealership was extremely unhelpful during repairs. On the bright side, the touchscreen display is big and navigation is super fast.”

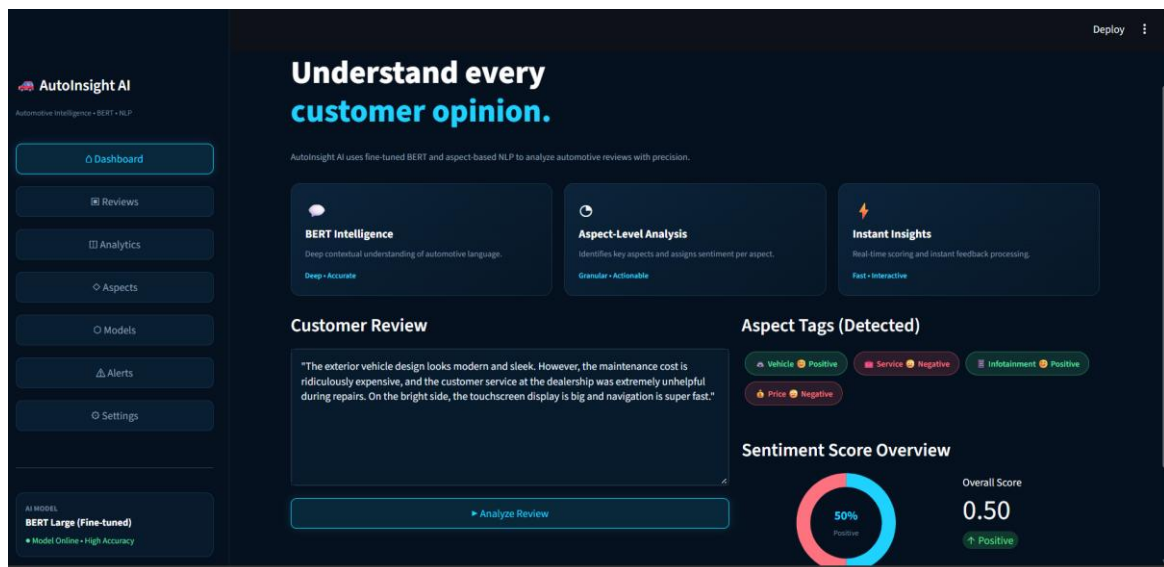


Figure 6.2: AutoInsight AI dashboard analysing a multi-aspect review

The dashboard detected four aspects and assigned sentiment as follows: **Vehicle – Positive**, **Service – Negative**, **Infotainment – Positive** and **Price – Negative**. These agree with the meaning of the review: the design and touchscreen are praised, while the service experience

and cost are criticised. With two of four aspects positive, the donut chart shows 50% positive and the overall score is 0.50. Two limitations are visible: the sentence about “maintenance cost” contributes to Price and (through the word “maintenance”) to Service, and the overall score counts each aspect equally.

6.6 Summary of Results

- The dataset is balanced and free of duplicated text across splits.
- The keyword extractor finds the labelled aspect in 88.0% of samples, with its main weakness being vocabulary coverage and the generic “car/vehicle” trigger.
- A TF-IDF + logistic regression baseline reaches 63.6% test accuracy, giving a reference which the fine-tuned BERT model is expected to exceed (Table 6.4).
- The end-to-end dashboard correctly separates positive and negative aspects within one multi-sentence review.

7. Discussion

7.1 Interpretation of Findings

The baseline results show that the task is not trivial even on short sentences: bag-of-words features confuse polarity when the same domain words appear in positive and negative sentences. A contextual model such as BERT can use word order and surrounding words (for example “not”, “too”, “higher than expected”) and the aspect prefix, and is therefore better suited to the problem. The aspect prefix approach is also economical: a single model handles all nine aspects instead of training nine separate classifiers.

7.2 Effect of Sentence-Level Context

In multi-sentence reviews, passing the entire text to the classifier for every aspect can blur opinions, because a strongly negative sentence about price may influence the prediction for an unrelated, positive aspect such as design. The dashboard reduces this by selecting only the sentences that mention the aspect’s keywords. The command-line `predict.py` in contrast passes the whole review, so short single-topic reviews are its best use case. Aligning the two paths is a straightforward improvement.

7.3 Limitations Observed

- **Dataset size and style.** The 216 samples are short, clean and templated. High scores on such data can overestimate real-world performance, and 33 test samples give a wide confidence interval.
- **Keyword-based aspects.** Detection depends on a fixed dictionary; synonyms, spelling variants and implicit aspects (“it never starts on cold mornings” refers to the engine) are missed. In `app.py` the keywords are matched as substrings, which can cause false positives (for example, “car” inside “card” or “scarce”); the word-boundary matching used in `aspect_extractor.py` avoids this and is recommended for both.
- **Overlapping vocabulary.** Words such as “cost”, “maintenance” and “efficient” can belong to several aspects (Price, Service, Mileage, Battery).
- **Rule-based cross-check.** The lexicon used in the dashboard is small and hand-made. Its negation rule (“not”/“no”) is deliberately simple and can flip sentiment incorrectly for phrases such as “no issues”. Its use as an override should be evaluated on held-out data before claiming improved accuracy.
- **Overall score.** The donut chart score is the share of positive aspects and treats all aspects as equally important; it does not reflect intensity or the model’s confidence.

- **Model naming.** The fine-tuned model is bert-base-uncased; the dashboard should describe it accordingly.

7.4 Practical Implications

For manufacturers, aspect-level scores let a team prioritise: for instance, a high volume of negative Service opinions calls for dealer training, while negative Battery opinions call for engineering review. For buyers, aspect tags make it possible to check quickly whether a model is strong in the areas they care about. The IVI-related aspect (Infotainment) is of particular interest because usability problems in touchscreens and navigation directly affect the driver experience.

8. Conclusion and Future Scope

8.1 Conclusion

This project developed an end-to-end system for aspect-based sentiment analysis of automotive reviews. A 216-sample labelled dataset covering nine aspects and three sentiment classes was created and processed; a bert-base-uncased model was fine-tuned on aspect-prefixed inputs; a keyword-based extractor was built to detect aspects; and an interactive Streamlit dashboard, AutoInsight AI, was implemented to present aspect-wise sentiment and an overall score. Evaluation showed that the aspect extractor recovers the labelled aspect in 88.0% of samples and that a classical TF-IDF baseline achieves 63.6% accuracy on the test split, providing a benchmark for the BERT model. The dashboard correctly distinguished positive and negative aspects within a single realistic review.

The work demonstrates that a compact pipeline built from widely available open-source tools can turn unstructured customer opinions into structured, actionable insight, and it provides a base which can be extended to larger and more realistic data.

8.2 Future Scope

- **Larger real-world dataset.** Collect and annotate thousands of reviews from car forums, dealer sites and app stores, including long multi-sentence reviews, and report results with confidence intervals or cross-validation.
- **Learned aspect extraction.** Replace the keyword dictionary with token-level sequence labelling (BERT-based aspect term extraction) or zero-shot classification to capture synonyms and implicit aspects.
- **Joint aspect-sentiment models.** Explore BERT variants such as sentence-pair inputs, domain-adapted or larger models (RoBERTa, DeBERTa), and compare against the baseline with a proper ablation of the lexicon cross-check.
- **Model confidence and calibration.** Show probabilities in the dashboard and weight the overall score by confidence and aspect importance.
- **Analytics and alerts.** Complete the Reviews, Analytics, Aspects, Models and Alerts pages with trend charts, aspect leaderboards, brand or model comparison and automatic alerts when negative sentiment rises.
- **Data ingestion and deployment.** Connect to review sources through APIs, add a REST API (for example, FastAPI), containerise with Docker and deploy to a cloud service.

- **Multilingual and code-mixed reviews.** Support Hindi, Bengali and code-mixed text, which is common among Indian car buyers, with multilingual transformer models.
- **Integration with in-vehicle systems.** As an extension, apply aspect-level feedback analysis to infotainment usability data collected from a real or simulated IVI system.

References

- [1] B. Liu, *Sentiment Analysis and Opinion Mining*. Morgan & Claypool Publishers, 2012.
- [2] M. Hu and B. Liu, “Mining and summarizing customer reviews,” in *Proc. 10th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2004, pp. 168–177.
- [3] C. J. Hutto and E. Gilbert, “VADER: A parsimonious rule-based model for sentiment analysis of social media text,” in *Proc. 8th Int. AAAI Conf. on Weblogs and Social Media*, 2014.
- [4] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems* 30, 2017.
- [5] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] M. Pontiki et al., “SemEval-2014 Task 4: Aspect based sentiment analysis,” in *Proc. 8th Int. Workshop on Semantic Evaluation (SemEval 2014)*, 2014, pp. 27–35.
- [7] C. Sun, L. Huang and X. Qiu, “Utilizing BERT for aspect-based sentiment analysis via constructing auxiliary sentence,” in *Proc. NAACL-HLT*, 2019, pp. 380–385.
- [8] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP 2020: System Demonstrations*, 2020, pp. 38–45.
- [9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems* 32, 2019.
- [10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] Streamlit Inc., “Streamlit documentation.” [Online]. Available: <https://docs.streamlit.io>
- [12] Hugging Face, “Transformers documentation.” [Online]. Available: <https://huggingface.co/docs/transformers>

Appendix

Appendix A: Sample Dataset Rows

Table A.1: Sample rows from processed_reviews.csv

Review	Aspect	Sentiment
The touchscreen is slow and unresponsive.	Infotainment	Negative
The price is higher than expected.	Price	Negative
The car provides poor protection.	Safety	Negative
The overall vehicle design is disappointing.	Vehicle	Negative
The model includes adjustable front seats.	Comfort	Neutral
The car has a standard cabin layout.	Comfort	Neutral
The car includes USB connectivity.	Infotainment	Neutral
The vehicle has a standard market price.	Price	Neutral
Engine response is quick and refined.	Engine	Positive
The display is bright and user friendly.	Infotainment	Positive
Fuel efficiency is very good.	Mileage	Positive
The vehicle performs efficiently in city traffic.	Mileage	Positive

Appendix B: aspect_extractor.py (Complete Listing)

```
import re

ASPECT_KEYWORDS = {

    "Vehicle": [
        "car", "vehicle", "model", "design", "body", "build"
    ],

    "Engine": [
        "engine", "motor", "power", "horsepower",
        "acceleration", "performance"
    ],

    "Battery": [
        "battery", "range", "charging",
        "charge", "battery life"
    ],

    "Mileage": [
        "mileage", "fuel economy", "consumption",
        "kmpl", "efficiency", "fuel efficient"
    ],

    "Safety": [
        "safety", "airbag", "brake", "braking",
        "abs", "adas", "collision", "protection"
    ],

    "Comfort": [
```

```

        "comfort", "comfortable", "seat", "seats",
        "ride", "suspension", "legroom", "cabin"
    ],

    "Service": [
        "service", "maintenance", "dealer",
        "repair", "workshop"
    ],

    "Infotainment": [
        "infotainment", "screen", "display",
        "navigation", "audio", "speaker",
        "bluetooth", "touchscreen"
    ],

    "Price": [
        "price", "cost", "expensive",
        "affordable", "value", "priced"
    ]
}

```

```
def extract_aspects(text):
```

```
    text = text.lower()
```

```
    found_aspects = []
```

```
    for aspect, keywords in ASPECT_KEYWORDS.items():
```

```
        for keyword in keywords:
```

```
            pattern = r"\b" + re.escape(keyword) + r"\b"
```

```
            if re.search(pattern, text):
```

```
                found_aspects.append(aspect)
```

```
                break
```

```
    return found_aspects
```

```
if __name__ == "__main__":
```

```
    test_reviews = [
```

```
        "The battery range is excellent.",
```

```
        "The engine is powerful but the mileage is poor.",
```

```
        "The seats are comfortable and the infotainment system is easy to use.",
```

```
        "The car is expensive but the safety features are excellent."
    ]
```

```
]
```

```
for review in test_reviews:
```

```
    aspects = extract_aspects(review)
```

```
print("\nReview:", review)
print("Detected aspects:", aspects)
```

Appendix C: prepare_data.py (Complete Listing)

```
import pandas as pd
from sklearn.model_selection import train_test_split

INPUT_FILE = "data/processed_reviews.csv"

TRAIN_FILE = "data/train.csv"
VALIDATION_FILE = "data/validation.csv"
TEST_FILE = "data/test.csv"

def main():

    print("Loading processed dataset...")

    df = pd.read_csv(INPUT_FILE)

    print("Original shape:", df.shape)

    # Create BERT input
    df["bert_text"] = (
        df["aspect"] + ": " + df["clean_review"]
    )

    # Convert sentiment labels to numbers
    label_mapping = {
        "Negative": 0,
        "Neutral": 1,
        "Positive": 2
    }

    df["label"] = df["sentiment"].map(label_mapping)

    # First split:
    # 70% training
    # 30% temporary
    train_df, temp_df = train_test_split(
        df,
        test_size=0.30,
        random_state=42,
        stratify=df["label"]
    )

    # Split temporary data:
    # 15% validation
    # 15% testing
    validation_df, test_df = train_test_split(
        temp_df,
        test_size=0.50,
        random_state=42,
        stratify=temp_df["label"]
    )

    # Save datasets
    train_df.to_csv(TRAIN_FILE, index=False)
    validation_df.to_csv(VALIDATION_FILE, index=False)
```

```

test_df.to_csv(TEST_FILE, index=False)

print("\nDataset preparation completed!")

print("\nTraining:", train_df.shape)
print("Validation:", validation_df.shape)
print("Testing:", test_df.shape)

print("\nLabel mapping:")
print(label_mapping)

print("\nExample BERT input:")
print(train_df[["bert_text", "sentiment", "label"]].head())

if __name__ == "__main__":
    main()

```

Appendix D: train_bert.py (Training Configuration and Trainer)

```

# -----
# 9. TRAINING CONFIGURATION
# -----

training_args = TrainingArguments(

    output_dir=OUTPUT_DIR,

    learning_rate=2e-5,

    per_device_train_batch_size=8,

    per_device_eval_batch_size=8,

    num_train_epochs=3,

    weight_decay=0.01,

    eval_strategy="epoch",

    save_strategy="epoch",

    load_best_model_at_end=True,

    metric_for_best_model="f1",

    greater_is_better=True,

    logging_steps=10,

    report_to="none",

    use_cpu=True
)

# -----
# 10. TRAINER
# -----

```

```

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=train_dataset,

    eval_dataset=validation_dataset,

    processing_class=tokenizer,

    data_collator=data_collator,

    compute_metrics=compute_metrics
)

# -----
# 11. START TRAINING
# -----

print("\n=====")
print("Starting BERT training...")
print("=====\n")

trainer.train()

```

Appendix E: predict.py (Inference Functions)

```

def predict_sentiment(aspect, review):

    text = aspect + ": " + review

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=128
    )

    with torch.no_grad():

        outputs = model(**inputs)

        prediction = torch.argmax(
            outputs.logits,
            dim=1
        ).item()

        return LABEL_NAMES[prediction]

def analyze_review(review):

    aspects = extract_aspects(review)

```

```
results = []

for aspect in aspects:

    sentiment = predict_sentiment(
        aspect,
        review
    )

    results.append({
        "aspect": aspect,
        "sentiment": sentiment
    })

return results
```

Appendix F: requirements.txt

```
pandas
numpy
scikit-learn
matplotlib
seaborn
transformers
datasets
torch
accelerate
streamlit
nltk
spacy
openpyxl
```